



11. Multiple Imputation by Chained Equation (MICE)

Predict missing values by learning from the rest of the data.

Previous Section:

 [10. Feature Selection](#)

Contents:

[1. What is the *Multiple Imputation by Chained Equations* Method?](#)

[2. Chained Equations Workflow](#)

[4. Example Workflow](#)

[5. Chained Equations with Python](#)

[Summary](#)

[References](#)

So far, we have explored simple techniques for handling missing values, such as replacing them with the mean, median, or mode. While these methods are fast and easy to apply, they often ignore the relationships between variables and may introduce bias into the dataset.

In this section, we introduce **Multiple Imputation by Chained Equations (MICE)**, one of the most advanced and widely used missing value imputation techniques in Exploratory Data Analysis (EDA).

Unlike previous methods, MICE trains a machine learning model to estimate missing values based on the other features in the dataset. But instead of performing this prediction only once, the process is repeated multiple times, allowing the imputed values to be continuously refined until they become more stable and reliable.

1. What is the *Multiple Imputation by Chained Equations* Method?

Multiple Imputation by Chained Equations (MICE) is a statistical technique for handling missing data by learning the relationships between variables rather than replacing missing values with a single fixed statistic.

- MICE predicts the missing values of one feature using the remaining features in the dataset. A machine learning model is trained to estimate the missing entries, and this process is repeated for each feature containing missing values.
- Rather than stopping after one prediction, MICE performs multiple rounds of imputation. After each round, the newly estimated values are used to retrain the models for the other variables, gradually improving the quality and consistency of the imputed data.
- Because the predictions are refined through several iterations, MICE generally produces more realistic imputations than simple methods such as mean, median, or mode replacement, especially when the variables in the dataset are strongly correlated.

2. Chained Equations Workflow

The MICE algorithm consists of three main stages. Instead of predicting every missing value at once, it repeatedly trains models for each variable with missing data, gradually refining the imputed values until they become stable.

Step 1. Initialization

Since machine learning models cannot be trained with missing values, MICE first creates a complete dataset using simple imputation methods.

- Identify all columns containing missing values.
- Replace missing values with simple estimates.
 - Numerical variables → Mean or Median
 - Categorical variables → Mode

- This serves only as the initial starting point for the algorithm.

Step 2. Chained Imputation

Once a complete dataset is available, MICE begins refining the imputed values one variable at a time.

- Select one column containing missing values.
- Temporarily remove its imputed values (set them back to missing).
- Train a machine learning model using all the remaining columns as input features.
 - Numerical variable → Regression model
 - Categorical variable → Classification model
- Predict the missing values for the selected column.
- Replace the temporary values with the model's predictions.
- Repeat the same procedure for every remaining column with missing values.

Step 3. Iterative Refinement

One complete pass through all variables is called an **iteration**.

- Repeat the chained imputation process multiple times.
- Each iteration uses the improved values generated during the previous iteration.
- Stop when:
 - Numerical predictions change very little between iterations, or
 - Categorical predictions remain unchanged.

By repeatedly learning the relationships among variables, MICE usually produces more realistic and reliable imputations than simple statistical replacement methods.

4. Example Workflow

Consider the following dataset containing several missing values.

Name	Age	Major	GPA
James	12	AI	3.5
Chris	NaN	CSC	3.2
Adam	15	NaN	2.8
Marian	NaN	AI	2.3
Peter	18	AI	2.1
Kevin	19	IT	NaN

Step 1. Initial Imputation

MICE first creates a complete dataset using simple statistical methods.

- **Age** → Replace missing values with the median age.
- **Major** → Replace missing values with the mode ("AI").
- **GPA** → Replace missing values with the median GPA.

The dataset now becomes:

Name	Age	Major	GPA
James	12	AI	3.5
Chris	16	CSC	3.2
Adam	15	AI	2.8
Marian	16	AI	2.3
Peter	18	AI	2.1
Kevin	19	IT	2.8

These values are only temporary starting estimates and will be refined during later iterations.

Step 2. Update the GPA Column

Suppose we begin by improving the **GPA** column.

- **2a. Reset the temporary GPA value**

Kevin's GPA is changed back to **NaN**.

Name	Age	Major	GPA
James	12	AI	3.5
Chris	16	CSC	3.2
Adam	15	AI	2.8
Marian	16	AI	2.3
Peter	18	AI	2.1
Kevin	19	IT	NaN

- **2b. Build the training dataset**

Use rows with known GPA values → **Features:** Age, Major; **Target:** GPA

Age	Major	GPA
12	AI	3.5
16	CSC	3.2
15	AI	2.8
16	AI	2.3
18	AI	2.1

A **regression model** is trained to learn the relationship between Age, Major, and GPA.

- **2c. Predict Kevin's GPA**

The trained model predicts Kevin's missing GPA using: Age = 19; Major = IT.

Suppose the model predicts: **Kevin's GPA = 2.5**. This replaces the temporary value.

Name	Age	Major	GPA
James	12	AI	3.5
Chris	16	CSC	3.2
Adam	15	AI	2.8
Marian	16	AI	2.3
Peter	18	AI	2.1

Name	Age	Major	GPA
Kevin	19	IT	2.5

Step 3. Update the Age Column

Next, MICE moves to the **Age** column.

- **3a. Reset the temporary age values**

Chris and Marian's ages are set back to **NaN**.

Name	Age	Major	GPA
James	12	AI	3.5
Chris	NaN	CSC	3.2
Adam	15	AI	2.8
Marian	NaN	AI	2.3
Peter	18	AI	2.1
Kevin	19	IT	2.5

Notice that Kevin's **updated GPA (2.5)** is now used instead of the initial statistical estimate.

- **3b. Build the training dataset**

Features: Major, GPA → **Target:** Age

Major	GPA	Age
AI	3.5	12
AI	2.8	15
AI	2.3	16
AI	2.1	18
IT	2.5	19

A **regression model** is trained to predict age.

- **3c. Predict the missing ages**

Using the trained model:

Chris (CSC, GPA = 3.2) → **Age = 14;**

Marian (AI, GPA = 2.3) → **Age = 17**

These predictions replace the temporary values.

Name	Age	Major	GPA
James	12	AI	3.5
Chris	14	CSC	3.2
Adam	15	AI	2.8
Marian	17	AI	2.3
Peter	18	AI	2.1
Kevin	19	IT	2.5

MICE continues in the same manner for every column containing missing values. After one complete pass, another iteration begins using the newly predicted values.

This process repeats until the predictions become stable, producing a more consistent and realistic completed dataset.

5. Chained Equations with Python

Scikit-Learn provides an implementation of the MICE algorithm through the **IterativeImputer** class. To better understand how it improves missing value imputation, we will compare it with two simpler techniques:

- **SimpleImputer** – replaces missing values using statistical measures such as the mean.
- **KNNImputer** – estimates missing values using the nearest neighboring samples.
- **IterativeImputer (MICE)** – repeatedly trains machine learning models to predict and refine missing values by learning the relationships among variables.

For this demonstration, we will generate a synthetic dataset containing **200 samples** and randomly introduce **30% missing values**. We will then apply each imputation method and compare their results.

- **Step 1. Import Libraries**

```
import numpy as np
import pandas as pd

from sklearn.impute import SimpleImputer, KNNImputer

# Enable IterativeImputer (experimental feature)
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
```

- **Step 2. Generate a Dataset**

```
np.random.seed(42)

n = 200

age = np.random.randint(18, 31, n)
height = 145 + age * 1.5 + np.random.normal(0, 4, n)
weight = 20 + height * 0.38 + np.random.normal(0, 5, n)
gpa = np.clip(2 + (31 - age) * 0.08 + np.random.normal(0, 0.25, n), 2.0, 4.0)

df = pd.DataFrame({
    "Age": age,
    "Height": np.round(height, 1),
    "Weight": np.round(weight, 1),
    "GPA": np.round(gpa, 2)
})

print(df.head(20))
```

- **Step 3. Introduce 30% Missing Values**


```
df_missing = df.copy()

mask = np.random.rand(*df_missing.shape) < 0.30
df_missing = df_missing.mask(mask)

print(df_missing.head(20))
```

```
# Check the number of missing values.
print(df_missing.isna().sum())
```

- **Step 4. Simple Imputation**

```
simple_imputer = SimpleImputer(strategy="mean")

df_simple = pd.DataFrame(
    simple_imputer.fit_transform(df_missing),
    columns=df_missing.columns
)

print(df_simple.head(20))
```

- **Step 5. KNN Imputation**

```
knn_imputer = KNNImputer(n_neighbors=5)

df_knn = pd.DataFrame(
    knn_imputer.fit_transform(df_missing),
    columns=df_missing.columns
)

print(df_knn.head(20))
```

- **Step 6. MICE (Iterative Imputation)**

```

mice_imputer = IterativeImputer(
    random_state=42,
    max_iter=10
)

df_mice = pd.DataFrame(
    mice_imputer.fit_transform(df_missing),
    columns=df_missing.columns
)

print(df_mice.head(20))

```

- Step 7. Compare the Results

```

comparison = pd.DataFrame({
    "Original": df["Height"],
    "Missing": df_missing["Height"],
    "Simple": df_simple["Height"],
    "KNN": df_knn["Height"],
    "MICE": df_mice["Height"]
})

print(comparison.head(20))

```

```

# You can also verify that all missing values have been filled.
print(df_missing.isna().sum())
print(df_simple.isna().sum())
print(df_knn.isna().sum())
print(df_mice.isna().sum())

```

Summary

In this section, we introduced **Multiple Imputation by Chained Equations (MICE)**, one of the most powerful techniques for handling missing data. Unlike simple imputation methods that replace missing values with a single statistic, MICE learns the relationships between variables by repeatedly training machine learning models to predict missing values.

We first learned the motivation behind MICE and how it differs from traditional imputation techniques. We then examined the **three-stage workflow** consisting of initialization, chained imputation, and iterative refinement. Through a step-by-step example, we saw how each variable is temporarily treated as the prediction target while the remaining variables are used to train a model, gradually producing more accurate imputations after each iteration.

Finally, we implemented MICE in Python using Scikit-Learn's **IterativeImputer** and compared it with **SimpleImputer** and **KNNImputer** on a dataset containing randomly generated missing values. While MICE is more computationally expensive than simpler methods, it often produces more realistic and reliable imputations because it captures the underlying relationships among features instead of relying on fixed statistical values.

References

<https://scikit-learn.org/stable/modules/impute.html>

<https://www.kaggle.com/code/leedonghyeok/multiple-imputation-mice>

Next Section:

 [12. Isolation Forest](#)